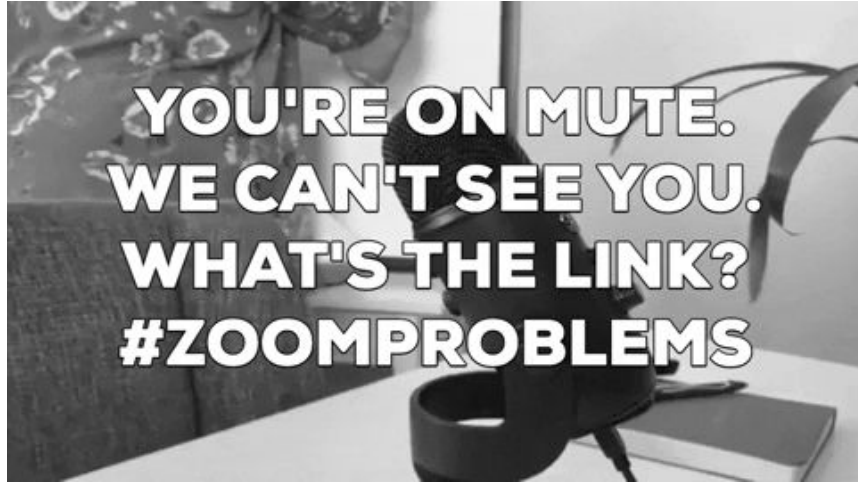


COMS W4156 Advanced Software Engineering (ASE)

September 3, 2024
First day of class!



Important Notice



All classes I teach are held on Zoom as part of my ADA (American Disability Act) [accommodations](#)

My courses still count as "in person" for international students who are required to take courses "in person"

If you do not want to take this course on Zoom, then drop the course. However, there is currently no non-Zoom version of this course

Are you on the Waitlist?

Student Services Online

► Wait List

As of this writing, the class is "full", but typically many slots open up during the program change period (between now and September 13)

As slots become available, I will admit students on the waitlist who responded to the "[background check](#)" in the order the responses were received. Students who do not respond will never be admitted to the class



Lecture Notes linked on [Courseworks Calendar](#)

"Live" links into google docs, so possibly changing up to and even during class

Pdfs will be posted on [Courseworks->Files->Lecture Notes](#) a few days later

Questions and Comments during Class

[Zoom](#), either voice or chat. Classes are recorded

Questions and Comments outside of Class

[Ed Discussion](#), supports "anonymous" posts

My office hours are held in another Zoom room immediately after class. Not recorded



Agenda

1. What I mean by 'Software Engineering'
2. Overview of this course

Software Developers after
fixing previous bugs with
with new bugs



What is Software Engineering?

Software Engineering !=
Programming

- *Time and Change* affect the sustainability of software, how the *codebase* adapts over the length of its life
- *Scale and Growth* affect the viability of software practices, how the *organization* adapts as it grows and evolves



Time and Change



Google search was originally a [graduate student project](#) called [Backrub](#), written in 1996 in Java and Python

In 1996, Java version 1.0, Python version 1.4

As of this writing, Java version [22](#), Python version [3.12](#)

The original Backrub *could not run today!*

It is unlikely that any other 1996 or even 2006 program written in Java or Python could run as-is today because of *external change*

The platform and execution environment has changed even if the program has not



Software Sustainability

Different from sustainable computing (aka [green computing](#)), although a mandate for green computing would count as an external change

How *painful* is it to modify the software to fit external change?

Could be change in ecosystem: language, compiler, libraries, operating system, hardware

Could be changes in regulations (e.g., EU's [GDPR](#) and other privacy protections)

Could be transient changes due to world circumstances (e.g., [sudden dramatic increase in unemployment insurance claims](#) early in pandemic)

Success depends on organizational culture, process and tools



Software Sustainability 2

The codebase has to be modified to fit changed ecosystem, new regulations, the world as is it today and tomorrow rather than when version 1 was released

How do we know whether the software still does what it is supposed to do, how do we keep it working the way it is supposed to, how do we sustain its core functionality?

We test the code to detect the bugs (any deviation from intended core functionality is a "bug") and we fix the bugs

And since fixing the code often introduces new bugs, we **test** the code again to detect the new bugs and then we fix the new bugs. And we keep modifying the code and we keep **testing** the code and we keep fixing more and more bugs



Exceptions

A few ancient programs survive

But their code never changes and they are (mostly) exempt from external change



Scale and Growth

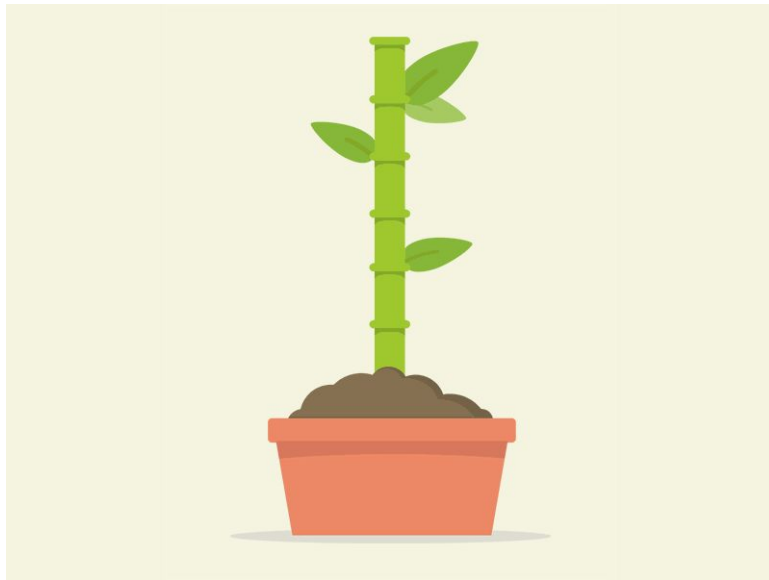
Backrub was written by Larry Page, Sergey Brin and some other Stanford graduate students

Their codebase was probably around 10k LOC

At least 25k software engineers now work for Google

Current Google codebase contains billions LOC - but its still (mostly) in a single monolithic codebase (all the developers can access all the code)

Elaborate processes, tools and technology enable scaling of both codebase and organization



Human Scalability Reality Check



For course programming assignments where two people work together, it might be feasible to email new versions of the code back and forth

This usually fails by the time the team grows to four people - no one knows what is the current version or who has it

For course projects where four or five people work together using a shared [github](#) (or similar) version control repository, it might be reasonable to turn on [notifications](#) to automatically send email to everyone in the team when anyone pushes modified code

But imagine every software engineer at Google getting email whenever any other software engineer at Google pushes some code (16k commits per day!)

	COMMENT	DATE
○	CREATED MAIN LOOP & TIMING CONTROL	14 HOURS AGO
○	ENABLED CONFIG FILE PARSING	9 HOURS AGO
○	MISC BUGFIXES	5 HOURS AGO
○	CODE ADDITIONS/EDITS	4 HOURS AGO
○	MORE CODE	4 HOURS AGO
○	HERE HAVE CODE.	4 HOURS AGO
○	AAAAAAA	3 HOURS AGO
○	ADKJFSJKDFJSOKLFT	3 HOURS AGO
○	MY HANDS ARE TYPING WORDS	2 HOURS AGO
○	HAAAAAAAANDS	2 HOURS AGO

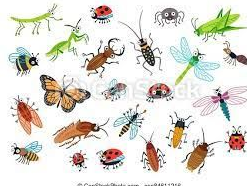
AS A PROJECT DRAGS ON, MY GIT COMMIT MESSAGES GET LESS AND LESS INFORMATIVE.

Human Scalability Reality Check 2



Imagine every change in the codebase potentially impacting the core functionality of every other part of the other code ... simultaneously being changed by other software engineers

Imagine all the bugs!



How to Learn Sustainable and Scalable SE

Ideally, the entire class would work on finding and fixing bugs in 25+ year old software consisting of a billion files, with new features constantly being added by tens of thousands of other developers, with four billion active users ...

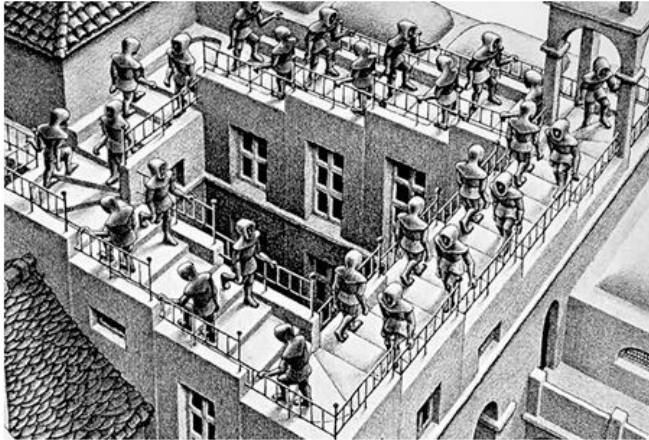
But that's not going to happen

What *will* happen...



Agenda

1. What I mean by 'Software Engineering'
2. Overview of this course



Overview of COMS W4156 Advanced Software Engineering

Themes:

- Building Useful Software
- Finding and Fixing Bugs

Workload:

- Individual Miniproject
- Team Project
- Assessments
- Class Participation

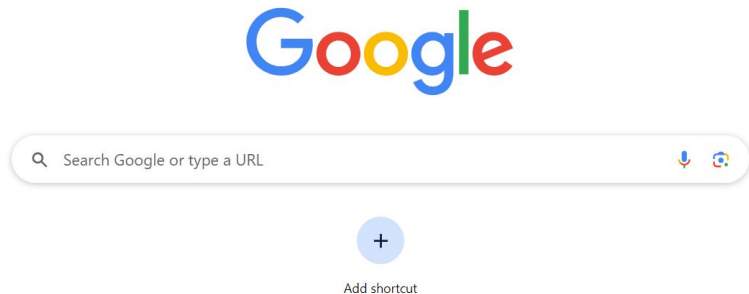
Also some reading assignments
(all freely available online, you do not
need to purchase any textbooks)



Building Useful Software

You probably think of Google Search as an App for users

With a UI that looks like this:



But Google Search is also a Service for developers

With an API that looks like this:

API operations

There is only one method to invoke in the Custom Search JSON API:

Operation	Description	REST HTTP mapping
list	Returns the requested search results from a Programmable Search Engine.	GET

API data model

The result of a search query to the Custom Search JSON API is a JSON object that includes three types of data:

- Metadata describing the requested search (and, possibly, related search requests)
- Metadata describing the Programmable Search Engine
- Search results

What is a Service?

Service != App



A service is software useful for *many* Apps, not just one

A library is also software useful for many Apps, but a library is statically or dynamically linked with each App and runs together with App on same machine. A service runs on some machine on the network, independent of any App, and Apps anywhere on the network connect to the same service (Service != Library)

Both a service and a library provide an application programming interface ([API](#)) for code, not a user interface (UI) for human users (API != UI)

Services, libraries and the differences between services, libraries and apps will be explained in more detail next week

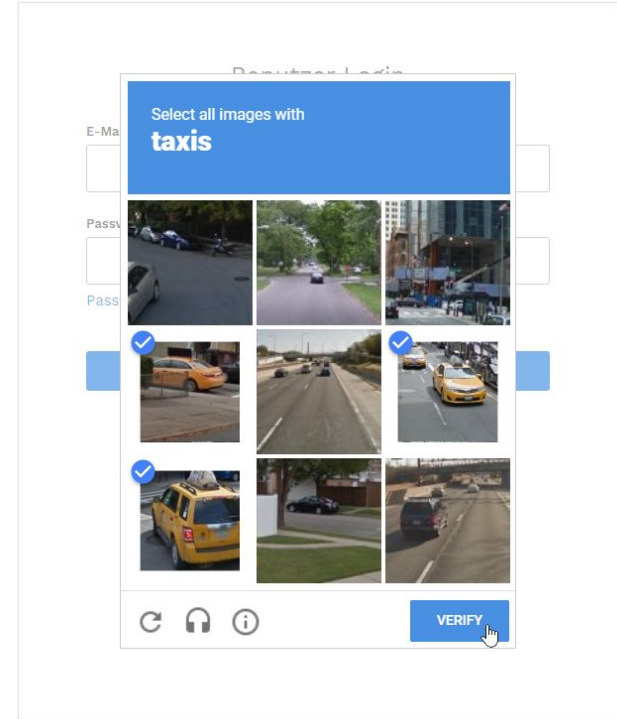
What is an API?



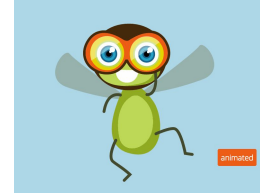
You have seen widgets like these in the UIs of many different unrelated Apps

Do you think every App developer coincidentally implements exactly the same thing, writing the code on their own from scratch?

No, of course not! Everyone is using the same reCaptcha code. Those developers' Apps include calls to that code. The set of supported calls is [reCaptcha's API](#)



Finding and Fixing Bugs



Testing

Style Checking

Static analysis

Dynamic analysis

Testing

Testing

Testing

Testing

Testing

If you hate testing (most students do!), and do not want to learn about testing (most students don't!), don't complain - just drop the course

Warning: Your first 5-10 years as a software engineer will almost certainly include testing other people's code (and fixing bugs in other people's code)

Testing: Seek And Ye Shall Find

Real-world codebases are modified repeatedly to fit changing ecosystem, changing regulations, the world as is it today, tomorrow, next year, next decade rather than when version 1 was released

Most software organizations maintaining real-world codebases periodically release new features as well as new bug-fix patches

How do we know whether the software still does what it's supposed to do, how do we keep it working the way it's supposed to, how do we sustain its core functionality?



We **test** the code to detect the bugs and we fix the bugs

And since fixing the code often introduces new bugs, we **test** the code to detect the new bugs and we fix the new bugs

And we keep modifying the code and we keep **testing** the code and we keep fixing more and more bugs

If software does not have any more bugs to fix, that means no one is using it

Individual Miniproject (20%)

Three parts: finding and fixing bugs in existing code (we provide two versions of this code, Java and C++), adding a feature, and automating build/test/deploy

Intended as small-scale "practice" for the team project

The assignments are posted in courseworks and we will discuss further after everyone has had a chance to look at those assignments

In the meantime, If you have questions about the miniproject assignments, post them publicly on Ed Discussion, so other students can see the answer, do not send email



Team Project (50%)

Ideally 4 students per team, 5 students ok
(no teams of 1, 2, 3, 6, 7, 8, 9, ...)



Students arrange their own teams, use [Ed Discussion](#) or [Discord](#) (and/or anything else you want) to find team members. If time permits, I'll run breakout rooms to help students meet other students in the class

Do [Coping with Teams](#) reading assignment *before* finalizing your team

Every team will have a Mentor who is either an IA or a Leader for this course

Team projects will develop and test a **service** of their own choice, within constraints

Team Project Constraints



The service must be a service, not a library and not an app

The service must be developed in Java or C++

Python not allowed (except for AI components, if interested talk to me *in advance*)

The first iteration will focus entirely on the service, the second iteration will add a client that uses the service's API

The client depends on the service but the service must not depend on the client

The client must target an acceptable user group that is not university students, staff or faculty (except for individuals who happen to also be members of an acceptable user group)

What Is An Acceptable User Group?

- Elderly (defined for this purpose as age 75 or over)
- Disabled or impaired (e.g., vision, hearing, speech, motor, cognitive)
- Healthcare workers (doctors, medical technicians, nurses, aides, etc.)
- First responders (police, fire, EMT)
- Indigenous peoples ("distinct social and cultural groups that share collective ancestral ties to the lands and natural resources where they live, occupy or from which they have been displaced")
- Homeless people ("lacks a fixed, regular, and adequate nighttime residence")
- Maybe others - ask me *in advance*



Team Project Milestones

TEAM FORMATION

PROJECT PROPOSAL



(meet with Mentor)

FIRST ITERATION

FIRST ITERATION DEMO (for Mentor)

SECOND ITERATION

SECOND ITERATION DEMO (for Mentor)

DEMO DAY

(for me and any students in class who want to watch other teams' demos)

“Differential Grading”



Default is for all members of the same team to get the same grade for the project

But every year there are a few students who do not do their fair share

Such students will receive a lower grade than their teammates, possibly a *much* lower grade, based on input from their other team members and their team Mentor

Differential grading is applied overall at end of semester, not per assignment

Do [Coping with Teams](#) reading assignment *before* finalizing your team!

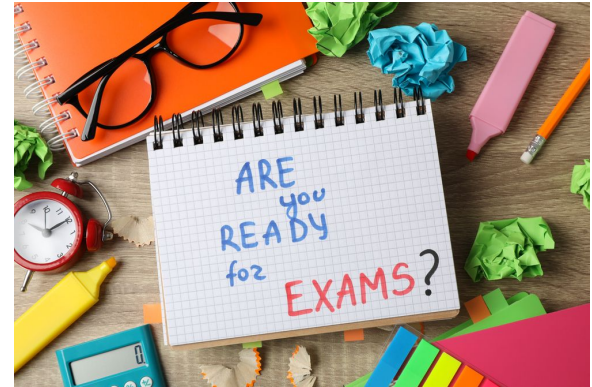
Assessments (20%)

Two individual assessments, one after the first iteration of team project and the other after the second iteration, each worth 10% of total grade

"Open Book" but you must work alone, not with your team, not with anyone else

Essentially a take-home exam but not timed, you will get four days to work on it and can spend as much or as little time as you want

We expect each assessment to take four to six hours, so if you finish in significantly less time, you probably did something "wrong". Check the questions and your answers carefully!



Class Participation (10%)

10% matters = difference between A and B!

Half (5%) based on attendance at Zoom class sessions after program change period ends. We will use Zoom's built-in reports to determine who attended and for how long (at least an hour, unless class ends early).

So make sure your Zoom name is the same as your courseworks name. If you are disconnected and reconnect, use the same name!

CVN students who cannot attend Zoom class sessions should do the "[CVN-only](#)" assignment for each session they miss

Non-CVN students can do the CVN-only assignment **only** if they have an acceptable (to me) excuse. Example acceptable: religious observance, illness or injury sufficiently severe that you cannot attend on Zoom, job interview. Example unacceptable: Forgot, slept late, busy



Class Participation (10%) part 2

Other half (5%) based on saying something non-trivial during class and/or posting something non-trivial on [edstem](#) *throughout semester*, not just once

Example trivial: "yes", "no", "thank you", "me too", "when is xxx due?", "can I do something that you already said is not allowed?", etc.

During Zoom class sessions, you can talk using your microphone and/or you can type public messages into Zoom chat. Zoom records both, make sure to use your courseworks name!

Outside class, you can post something of likely interest to other students in the class. Must be **public** post that other students can see and comment on. Again, use your courseworks name! CVN students should post *in addition to* their [CVN-only](#) assignments (which only count for attendance)



Next Class

Finish anything we didn't get to today

Version Control Overview

Github demo

if time permits: Miniproject Overview



Lecture Notes linked on [Courseworks Calendar](#)

"Live" links into google docs, so possibly changing up to and even during class

Pdfs will be posted on [Courseworks->Files->Lecture Notes](#) a few days later

Questions and Comments during Class

[Zoom](#), either voice or chat. Classes are recorded

Questions and Comments outside of Class

[Ed Discussion](#), supports "anonymous" posts

My office hours are held in another Zoom room immediately after class. Not recorded



Are you on the Waitlist?

Student Services Online

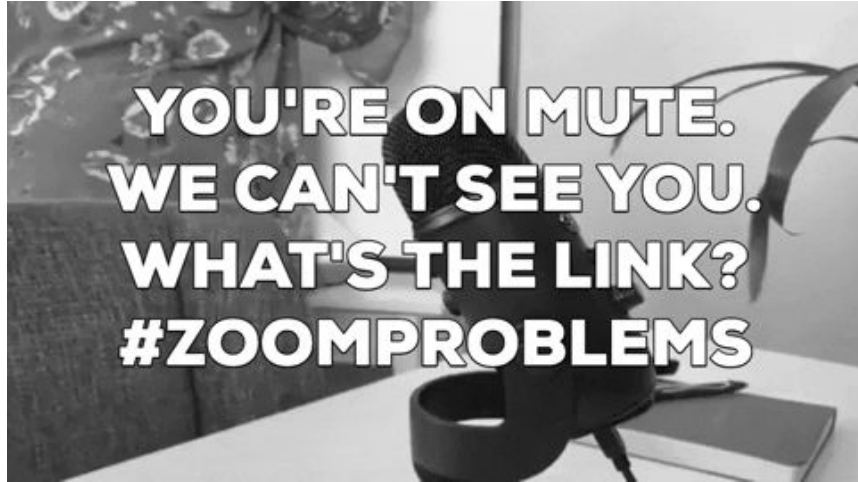
► Wait List

As of this writing, the class is "full", but typically many slots open up during the program change period (between now and September 13)

As slots become available, I will admit students on the waitlist who responded to the "[background check](#)" in the order the responses were received. Students who do not respond will never be admitted to the class



Important Notice



All classes I teach are held on Zoom as part of my ADA (American Disability Act) [accommodations](#)

My courses still count as "in person" for international students who are required to take courses "in person"

If you do not want to take this course on Zoom, then drop the course. However, there is currently no non-Zoom version of this course

Ask Me Anything